Lab 4. To-Do-List 앱 개발

Gemini CLI 활용하기

1) Gemini CLI 기본 명령어

Gemini CLI의 핵심 특징

- 터미널 네이티브 : 새로운 IDE나 확장프로그램 설치 불필요
- 전체 코드베이스 이해 : 프로젝트 구조와 컨텍스트를 자동으로 파악
- 직접 액션 실행 : 파일 편집, 명령어 실행, Git 작업까지 자동화
- MCP 지원 : Google Drive, Slack 등 외부 도구와 연동
- Gemini 2.5 Pro 기반 : 100만 토큰 컨텍스트 윈도우
- 관대한 무료 티어 : 분당 60회, 하루 1000회 무료 사용

기본명령어

명령어	설명	예시
gemini	대화형 모드 시작 (REPL)	gemini
gemini -p "질문"	질문 한 번 하고 바로 종료	Gemini -p "이 프로젝트 설명해줘"
cat 파일 | gemini	파일 내용을 파이프로 전달	cat logs.txt | gemini
gemini -i "질문" 또는 gemini "질문"	질문 실행 후 대화 계속	gemini -i "이 프로젝트 목적이 뭐야?"
gemini -r "latest"	가장 최근 세션 이어서 하기	gemini -r "latest"
gemini -r "latest" "질문"	최근 세션에 새 질문 던지기	gemini -r "latest" "타입 에러 확인해줘"
gemini -r "세션ID" "질문"	특정 세션 ID로 이어서 하기	gemini -r "abc123" "이 PR 마무리해줘"

주요 옵션

▪ 세션 관련

옵션	설명
--list-sessions	현재 프로젝트의 세션 목록 보기
--delete-session 번호	특정 세션 삭제

▪ 출력 관련

옵션	설명
--output-format / -o	출력 형식 (text, json, stream-json)
--screen-reader	스크린 리더 모드 (접근성)

자주 쓰는 gemini 명령 옵션

옵션	단축키	설명
--model	-m	사용할 모델 지정 (기본값: auto)
--prompt-interactive	-i	질문 실행 후 대화형 모드 유지
--sandbox	-s	AI가 내 파일을 직접 수정하지 못하도록 안전한 샌드박스 환경에서 실행
--resume	-r	이전 세션 이어하기 ("latest" 또는 세션 번호)
--debug	-d	디버그 모드 (상세 로그 출력)
--version	-v	버전 확인
--help	-h	도움말 보기

실습 준비

- VibeCoding 폴더에 Lab2 폴더 생성 후 이동
- Lab2 폴더에 다음 내용으로 error.log 파일 생성

```
2026-03-30 10:01: Error: Connection timeout at database:5432
2026-03-30 10:05: Warning: Memory usage high (85%)
2026-03-30 10:10: Error: Authentication failed for user 'admin'
```

기본 명령어 실습

```
gemini --version
cat error.log | gemini "이 로그 파일에서 발생한 에러들을 짧게 요약해줘"

gemini -p "안녕"
gemini -i "안녕" 또는 gemini "안녕"

# 대화 이어가기
조금 전의 @error.log 파일에서 두 번째 줄에 있는 Warning은 무시해도 되는 수준이야?
/quit
```

기본 명령어 실습

```
# 가장 최근 대화 강제로 불러오기
gemini -r "latest" "아까 그 로그 분석 내용에 이어서, 보안 관련 문제만 다시 요약해줄래?"
/quit

# 세션 목록 확인
gemini --list-sessions

# 특정 세션 ID로 이어서 요청하기
gemini -r "f6950f87-4a3f-49ba-b231-b728a53e4c61" -p "이 에러들에 대한 최종 보고서를 한 줄로
요약해줘"
```

기본 명령어 실습

```
# 세션 목록 확인
gemini --list-sessions

# 특정 세션 삭제
gemini --delete-session "150f431f-b19e-4f2b-b49b-2ec3e642ae08"

# 세션 목록 확인
gemini --list-sessions
```

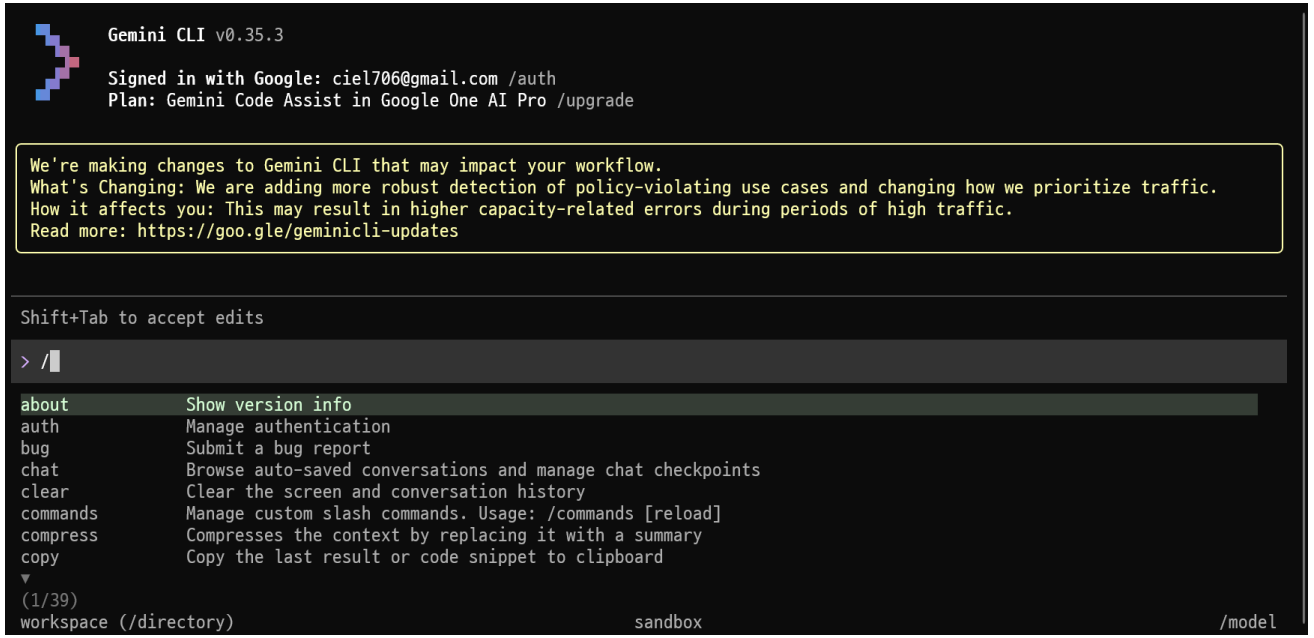
/(슬래시) 명령어 vs 자연어

- Gemini CLI는 두 가지 방식으로 명령을 내릴 수 있습니다.

[자연어]

```
# 파일 생성/수정
package.json에 새로운 의존성을 추가해줘
# 코드 분석
이 함수에서 성능 병목이 있는지 확인해줘
# 버그 수정
TypeError가 발생하는 이유를 찾아서 고쳐줘
```

[/명령어



기본 세션 및 시스템 명령어

- CLI의 실행 상태를 관리하고 도움말을 확인하는 가장 기초적인 명령어입니다.

명령어	설명	비고
/help	사용 가능한 모든 명령어와 단축키 목록을 표시합니다.	/?와 동일
/clear	터미널 화면과 현재 대화 문맥(Context)을 초기화합니다.	Ctrl+L 단축키
/quit	Gemini CLI를 종료합니다.	/exit와 동일
/stats	현재 세션의 토큰 사용량과 세션 지속 시간을 확인합니다.	비용 관리 시 유용
/about	현재 설치된 Gemini CLI의 버전 정보를 확인합니다.	버그 리포트 시 필요

기본 세션 및 시스템 명령어

```
> /about

About Gemini CLI

CLI Version                                    0.35.3
Git Commit                                     52d103469
Model                                          Auto (Gemini 3)
Sandbox                                        no sandbox
OS                                             win32
Auth Method                                    Signed in with Google (ciel706@gmail.com)
Tier                                           Gemini Code Assist in Google One AI Pro
```

기본 세션 및 시스템 명령어

```
> /stats

Session Stats

Interaction Summary
Session ID:                 c1db7db4-8c68-41a7-9e2d-8e6b39ed464d
Auth Method:                Signed in with Google (ciel706@gmail.com)
Tier:                       Gemini Code Assist in Google One AI Pro
Tool Calls:                 9 ( ✓ 9 x 0 )
Success Rate:               100.0%
User Agreement:             100.0% (9 reviewed)

Performance
Wall Time:                  14m 30s
Agent Active:               2m 53s
  » API Time:               1m 33s (53.7%)
  » Tool Time:              1m 20s (46.3%)
```

대화 및 문맥 관리 (세션 저장/복구)

- 진행 중인 대화 흐름을 저장하거나, 과거의 특정 시점으로 돌아갈 때 사용합니다.

명령어	설명	예시
/chat save <tag>	현재 대화 상태를 특정 태그로 저장합니다.	/chat save feature-a
/chat list	저장된 대화 로그(체크포인트) 목록을 보여줍니다.	-
/chat resume <tag>	저장했던 대화를 불러와 이어서 진행합니다.	/chat resume feature-a
/compress	긴 대화 내용을 요약본으로 대체하여 토큰을 절약합니다.	컨텍스트가 너무 길 때 사용
/restore	도구(Tools)에 의해 변경된 파일 상태를 이전으로 복구합니다.	--checkpointing 옵션 필요

대화 및 문맥 관리 실습

```
# 현재 저장된 대화 기록 목록 확인
/chat list

# 현재 대화 기록을 'my-project-feature' 태그로 저장
/chat save my-project-feature

# 저장된 'my-project-feature' 대화 기록 불러오기
/chat resume my-project-feature
```

파일 및 워크스페이스 제어

- 로컬 파일 시스템과 상호작용하며 프로젝트를 분석할 때 필수적입니다.

명령어	설명	비고
@<path>	특정 파일이나 디렉토리 내용을 프롬프트에 포함합니다.	@src/main.js
!<command>	Gemini 내부에서 직접 쉘 명령어를 실행합니다.	!git status
/directory add	워크스페이스에 특정 디렉토리를 추가하여 컨텍스트화합니다.	/directory add ./src
/memory show	GEMINI.md 등 프로젝트 지침 파일의 내용을 확인합니다.	장기 기억(Memory) 확인
/init	프로젝트 루트에 기본 GEMINI.md 설정 파일을 생성합니다.	프로젝트 가이드라인 설정

파일 첨부

Gemini CLI는 파이프(|) 외에 @ 기호를 사용한 파일 참조 기능 지원

```
gemini "이 파일 분석해줘: @error.log"
```

gemini 프롬프트가 시작됨

```
> 이 파일 분석해줘: @error.log

✓  ReadManyFiles Will attempt to read and concatenate files using patterns:

ReadManyFiles Result (Target Dir: D:\VibeCoding\Study02)

Successfully read and concatenated content from 1 file(s).

Processed Files:
 - error.log

✦ error.log 파일의 내용을 분석한 결과, 세 가지 주요 이슈가 발견되었습니다:

  1. 데이터베이스 연결 시간 초과 (10:01): database:5432 포트로의 연결이 실패했습니다. 네트워크 상태나 DB 서버의 가동
     여부를 확인해야 합니다.
  2. 높은 메모리 사용량 (10:05): 메모리 점유율이 85%에 도달했다는 경고입니다. 리소스 누수나 서버 사양 부족 가능성이
     있습니다.
  3. 인증 실패 (10:10): 'admin' 계정의 로그인 시도가 실패했습니다. 단순 오타일 수 있으나, 반복될 경우 보안 점검이
     필요합니다.

                                                                  ? for shortcuts
Shift+Tab to accept edits

> Type your message or @path/to/file
```

모델 선택 (-m)

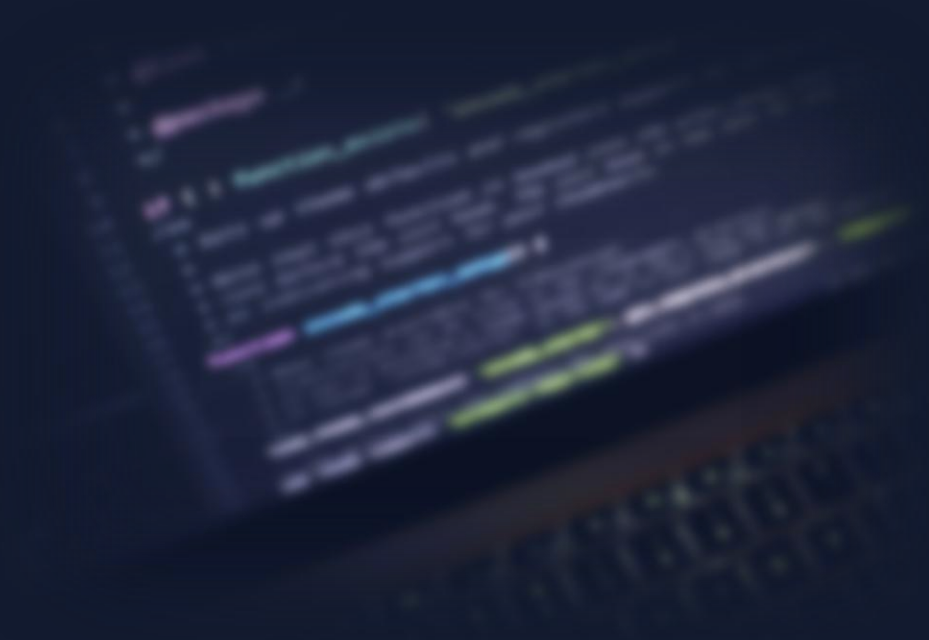
- 간편한 별칭(alias)을 사용하면 모델을 쉽게 지정할 수 있습니다.

별칭	실제 모델	용도
auto (기본값)	gemini-2.5-pro 또는 gemini-3-pro-preview	기본 설정, 프리뷰 기능 켜면 프리뷰 모델 사용
pro	gemini-2.5-pro 또는 gemini-3-pro-preview	복잡한 추론 작업용
flash	gemini-2.5-flash	빠르고 균형 잡힌 범용 모델
flash-lite	gemini-2.5-flash-lite	가장 빠름, 간단한 작업에 적합

- 사용예시

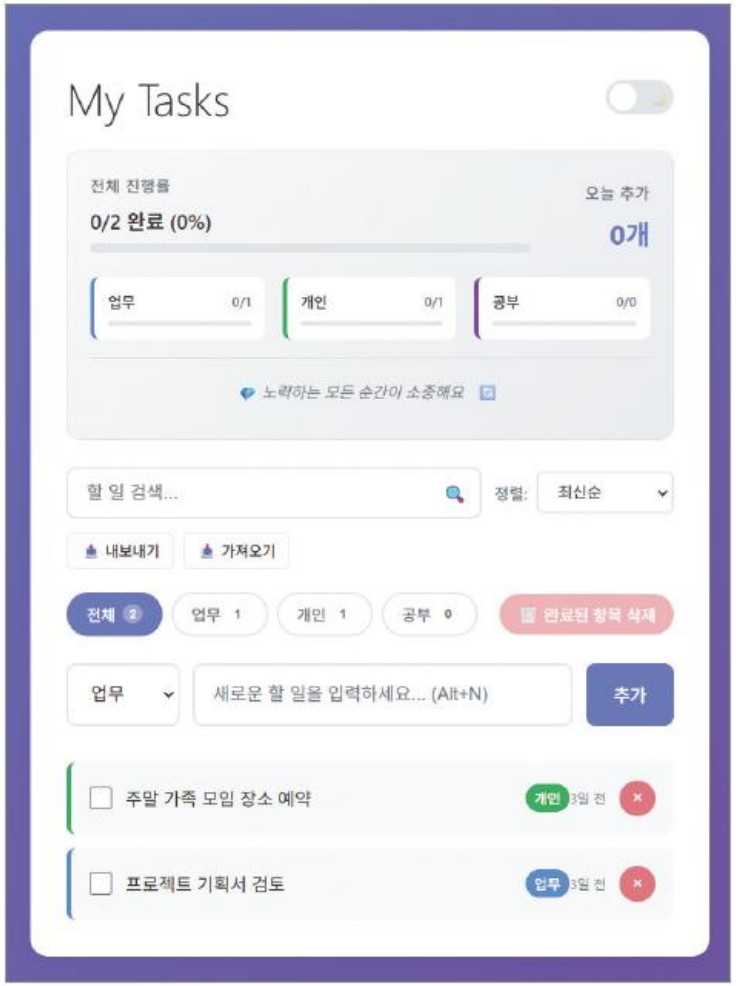
```
gemini -m flash "이 코드 리뷰해줘"
gemini -m pro "아키텍처 설계 도와줘"
```

2) To-Do-List 앱 제작



앱 제작 실습

- To-Do-List 애플리케이션 제작
  - 매일 해야 할 일을 간단히 기록하고 완료 여부를 체크
  - 업무/개인/공부와 같이 카테고리별로 항목을 분류해 관리
  - 앱은 웹 브라우저에서 바로 실행되며, 복잡한 프레임워크 대신 순수 HTML, CSS, 자바스크립트만으로 진행해 초보자도 바로 따라 할 수 있음



PRD 작성하기

- 웹 브라우저에서 Gemini에 접속하여 새로운 대화창에서 다음과 같이 프롬프트를 입력합니다.

너는 간결하면서도 개발자가 바로 작업에 착수할 수 있을 만큼 논리적인 문서 작성을 전문으로 할 수 있는 10년 차 IT 서비스 기획자이자 프로덕트 매니저(PM)야.

사용자가 매일 10~20개의 할 일을 관리하는 **할 일 (Todo List) 웹 앱**을 기획 중이야. 복잡한 서버 구축 없이 브라우저 내에서만 실행되며, 페이지를 새로고침해도 데이터가 유지되도록 localStorage를 활용할 계획이야. 기술 스택은 프레임워크 없이 순수 자바스크립트만 사용해줘.

아래 핵심 기능을 포함한 PRD(제품 요구사항 정의서)를 작성해줘.

- 할 일 추가, 수정, 삭제 기능
- 완료 체크(Status 관리) 및 시각적 표시
- 카테고리 분류 (개인/업무)
- 전체 진행률 표시 바(Progress Bar) 기능
- 데이터 영구 저장 (LocalStorage 연동)

PRD를 프롬프트로 변환하고 실행하기

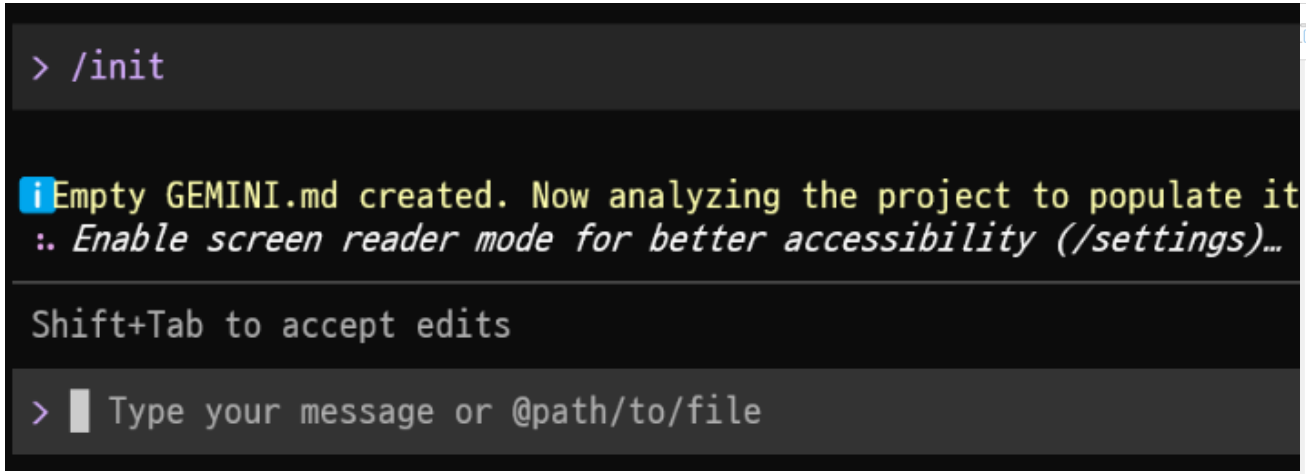
- 생성한 PRD를 바탕으로 Gemini CLI에서 실행할 수 있는 구체적인 프롬프트로 변환해 봅니다.

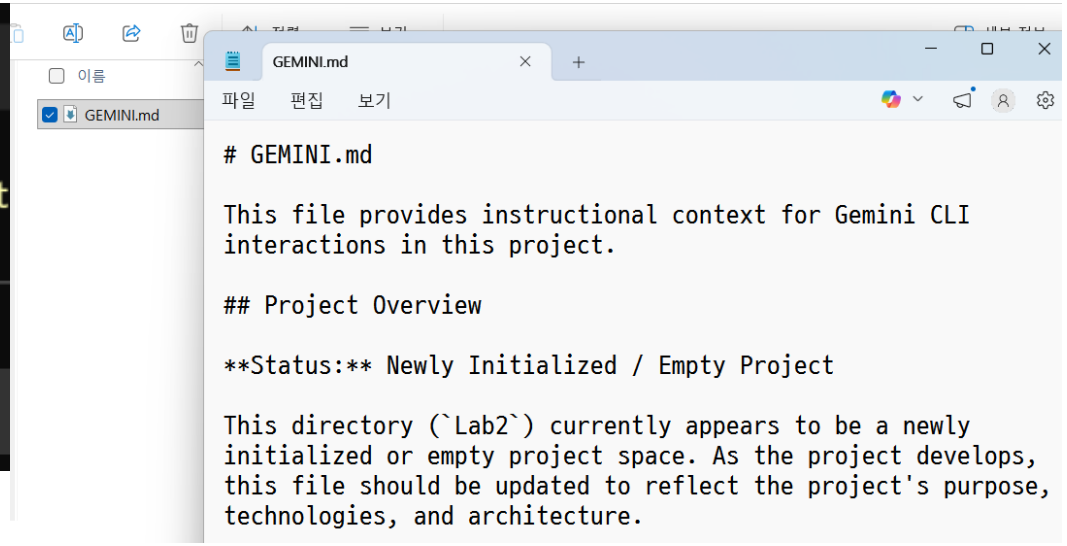
> 이 PRD를 Gemini CLI에서 사용할 단계별 프롬프트로 변환해 줘.
>
> 5개의 핵심단계로 정리해서 각 단계마다 정확한 지시사항을 만들어줘

- Gemini 웹에서 준비한 프롬프트를 단계별로 실행합니다. 텍스트를 복사해서 Gemini CLI에 붙여넣기 할 수 있습니다.

단계별 프롬프트 실행 : Gemini.md 파일 생성

- VibeCoding 폴더에 Lab4폴더를 생성한 후 해당 경로에서 터미널을 열고 gemini를 실행합니다.
- Gemini CLI가 실행되면 /init 명령어를 입력하여 프로젝트 초기 설정을 시작합니다. 현재 폴더를 분석하여 gemini.md 파일이 생성됩니다.(기본적인 프로젝트 설정 포함)





Gemini.md 파일 업데이트 요청

- Gemini.md 파일을 한글로 내용을 수정토록 요청

```
Gemini.md 파일의 내용을 한글로 다시 작성하고, 현재 프로젝트 진행에 맞춰 자동갱신해줘
```

```
16 + ## 실행 및 테스트 방법
17 + 1.  **실행**: `index.html` 파일을 최신 웹 브라우저(Chrome, Edge 등)에서 엽니다.
18 + 2.  **주요 기능 테스트**:
19 +     - 할 일 추가 (입력 후 'Add' 버튼 클릭 또는 Enter 키)
20 +     - 완료 상태 전환 (체크박스 클릭 시 취소선 적용)
21 +     - 할 일 삭제 ('X' 버튼 클릭)
22 +     - 데이터 보존 (페이지 새로고침 후에도 목록 유지 확인)
23 +
24 + ## 개발 규칙 및 컨벤션
25 + - **순수 자바스크립트**: 외부 라이브러리나 프레임워크 없이 바닐라 JS로 구현합니다.
26 + - **데이터 저장**: `{ text: string, completed: boolean }` 형태의 객체 배열을 `localStorage`에 JSON 문자열로
   저장합니다.
27 + - **UI 피드백**: 할 일 완료 시 취소선과 텍스트 색상 변경을 통해 시각적인 피드백을 제공합니다.
28 +
29 + ---
30 + *최종 업데이트: 2026-04-12*
```

GEMINI.md 파일의 내용을 한글로 작성하고, 현재 프로젝트 상태에 맞춰 업데이트를 완료했습니다.

앞으로도 프로젝트 내용에 변화가 생길 때마다 이 파일에 자동으로 반영하고, 항상 한글로 유지하도록 하겠습니다. 추가적인 요구사항이나 기능 수정이 필요하시면 언제든 말씀해 주세요!

단계별 프롬프트 예시(1)

- 1단계: 기본 구조 및 핵심 기능 구현
  - 단계별 프롬프트의 첫 번째 프롬프트(Step 1)를 복사해 Gemini CLI가 실행 중인 터미널에 붙여 넣기
  - 진행되는 내용을 확인하며 기다리면 곧 기본 구조가 구현되었다는 메시지가 나타남
  - 예시 ➔

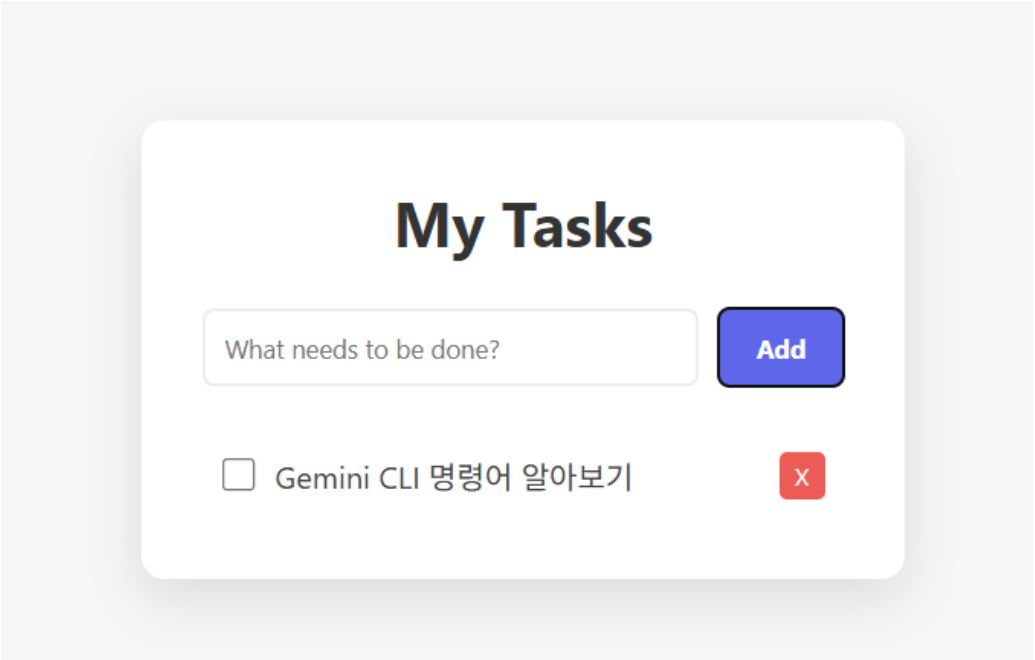
```
Step 1: 기본 구조 및 핵심 기능 구현
할 일 관리 앱의 기본 구조를 만들어줘.
요구사항:
1. index.html, style.css, script.js 세 개의 파일로
   구성
2. HTML 구조:
   • 앱 제목 "My Tasks"
   • 할 일 입력창 (input + 추가 버튼)
   • 할 일 목록을 표시할 컨테이너
3. JavaScript 기능:
   • 할 일 추가 (Enter 키와 버튼 클릭 모두 지원)
   • 할 일 삭제 (각 항목에 X 버튼)
   • 체크박스로 완료/미완료 토글
   • 완료 시 취소선 스타일 적용
4. localStorage에 데이터 저장:
   • 추가/삭제/완료 상태 변경 시 자동 저장
   • 페이지 새로고침 시 데이터 유지
5. 기본 CSS 스타일링:
   • 깔끔한 카드 형태의 레이아웃
```

단계별 프롬프트 예시(2)

- 'index.html'파일 실행 및 확인 요청

크롬을 실행해서 확인해줘

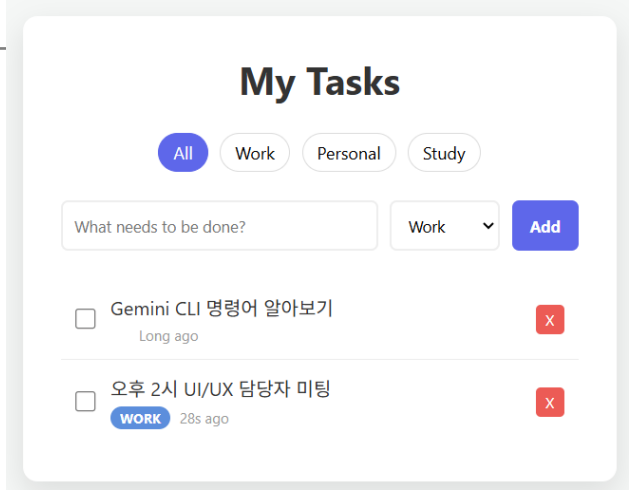
  - 기본 브라우저가 열리며, 할 일 관리 앱이 열림
  - 기능을 테스트하기 위해 [새로운 할 일을 입력하세요...] 입력창에 할 일을 추가해보고 확인



단계별 프롬프트 예시(3)

- 2단계: 카테고리 기능 및 UI 개선
  - 단계별 프롬프트의 두 번째 프롬프트(Step 2)를 복사해 Gemini CLI에 입력
  - 예시

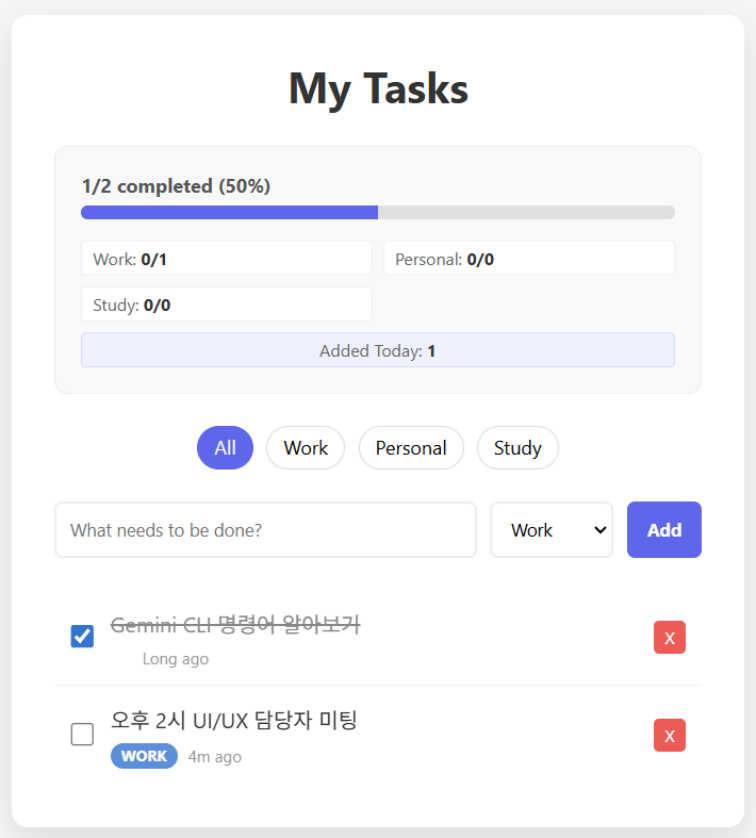
```
Step 2: 카테고리 기능 및 UI 개선
기존 코드에 카테고리 기능을 추가하고 UI를 개선해줘.
요구사항:
1. 카테고리 기능:
    • 3가지 카테고리: 업무(work), 개인(personal), 공부(study)
    • 할 일 추가 시 카테고리 선택 드롭다운
    • 각 할 일 항목에 카테고리 색상 태그 표시
    • 카테고리 필터 버튼 (전체/업무/개인/공부)
2. UI 개선:
    • 카테고리별 색상: 업무(파란색 #4A90E2), 개인(초록색 #27AE60), 공부(보라색 #8E44AD)
    • 필터 버튼을 상단에 배치
    • 선택된 필터 버튼 강조 표시
    • 할 일 항목에 생성 시간 표시 (예: "2시간 전")
3. 데이터 구조 업데이트:
    • category 필드 추가
```



단계별 프롬프트 예시(4)

- 3단계: 진행률 대시보드 추가
  - 단계별 프롬프트의 세 번째 프롬프트(Step 3)를 복사해 Gemini CLI에 입력

```
Step 3: 진행률 대시보드 추가
진행률 대시보드를 추가하고 인라인 수정 기능을 구현해줘.
요구사항:
1. 진행률 대시보드:
    • 앱 상단에 통계 섹션 추가
    • 전체 진행률: "5/10 완료 (50%)" 형식 + 프로그레스 바
    • 카테고리별 미니 진행률 표시 (각 카테고리 완료/전체)
    • 오늘 추가된 할 일 개수 표시
2. 인라인 수정 기능:
    • 할 일 텍스트 더블클릭 시 편집 모드
    • input 필드로 변경되어 수정 가능
    • Enter로 저장, ESC로 취소
    • 편집 중 카테고리도 변경 가능한 select 박스
3. UI 애니메이션:
    • 진행률 바 부드러운 전환 효과
    • 항목 추가/삭제 시 페이드 애니메이션
```

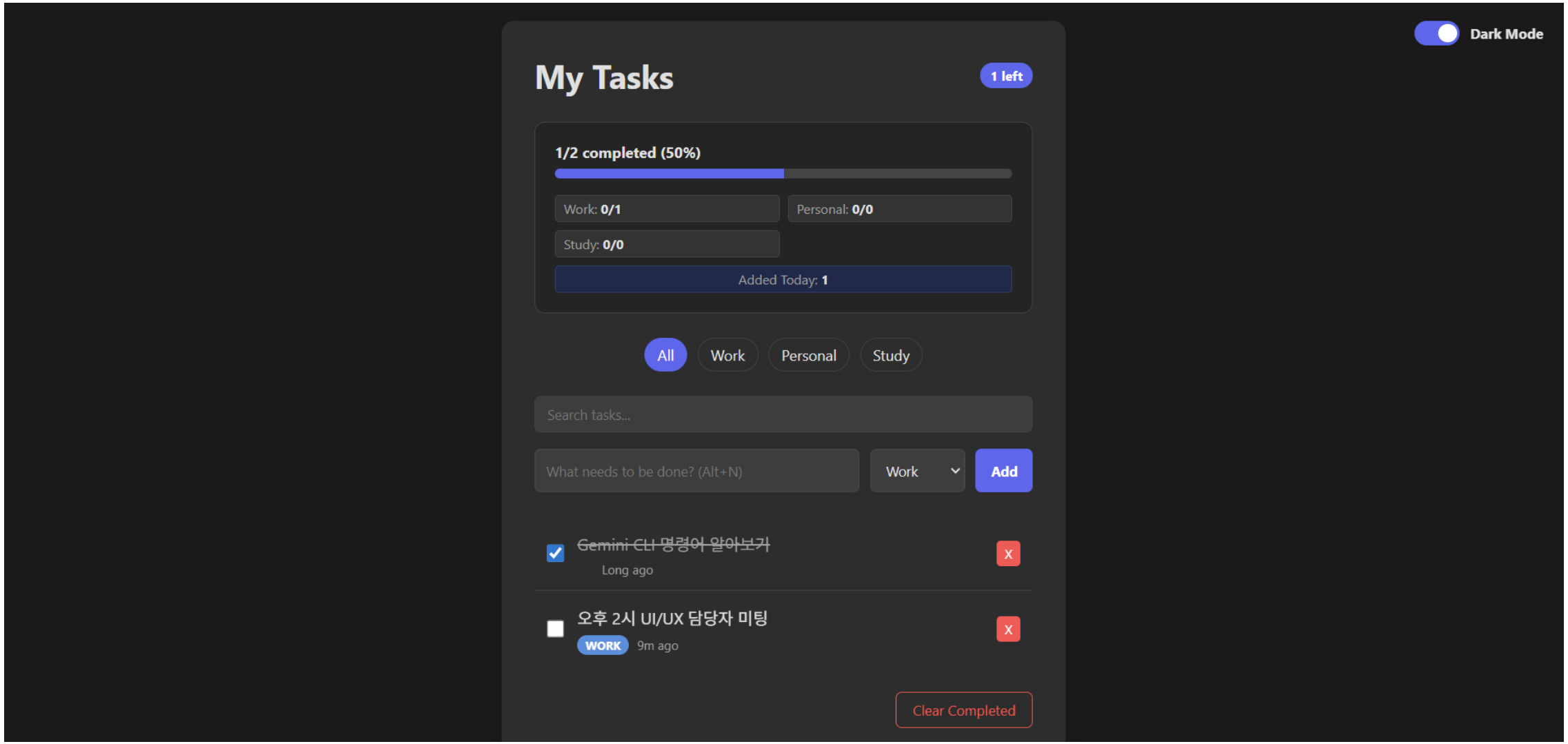


단계별 프롬프트 예시(5)

- 4단계: 다크 모드 및 고급 기능
  - 단계별 프롬프트의 네 번째 프롬프트(Step 4)를 복사해 Gemini CLI에 입력
    - 다크 모드를 구현하고 완료된 항목을 삭제하거나 단축키 사용 등의 기능 추가

```
Step 4: 다크 모드 및 고급 기능
다크 모드와 추가 기능들을 구현해줘.
요구사항:
1. 다크 모드:
    • 우측 상단에 다크/라이트 모드 토글 스위치
    • 다크 모드 색상: 배경 (#1A1A1A), 카드 (#2D2D2D), 텍스트 (#E0E0E0)
    • 선택한 테마 localStorage에 저장
    • 부드러운 전환 애니메이션
2. 추가 기능:
    • "완료된 항목 모두 삭제" 버튼 (확인 다이얼로그 포함)
    • 할 일 검색 기능 (실시간 필터링)
    • 남은 할 일 개수 배지 표시
    • 빈 상태 메시지 ("할 일이 없습니다. 추가해보세요!")
3. 키보드 단축키:
    • Alt+N: 새 할 일 입력창 포커스
    • Alt+1,2,3,4: 카테고리 필터 전환
```

4단계 완료 화면


Dark Mode
My Tasks
1 left
1/2 completed (50%)
Work: 0/1
Personal: 0/0
Study: 0/0
Added Today: 1
All
Work
Personal
Study
Search tasks...
What needs to be done? (Alt+N)
Work
Add
Gemini CLI 명령어 알아보기
Long ago
X
오후 2시 UI/UX 담당자 미팅
WORK
9m ago
X
Clear Completed

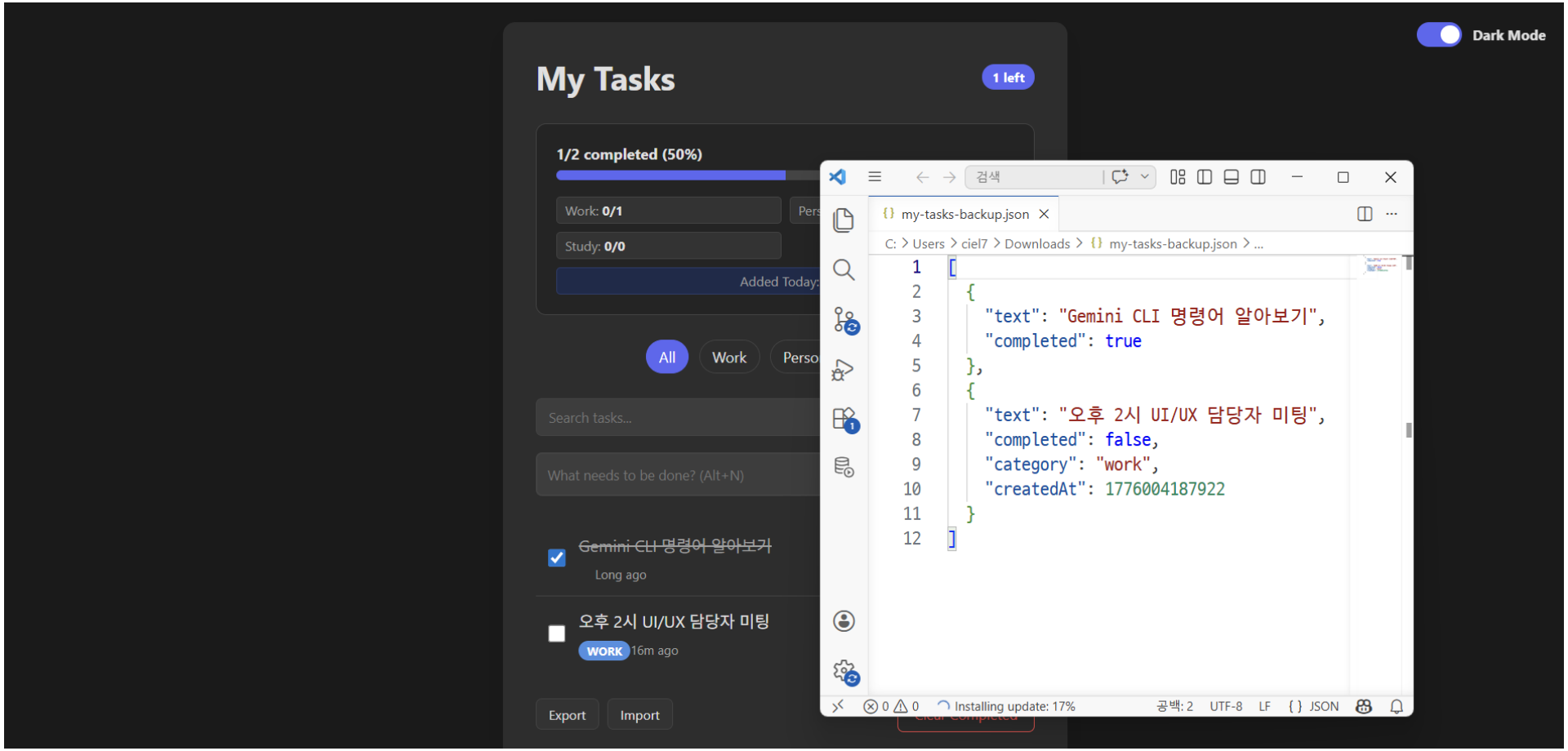
단계별 프롬프트 예시(5)

- 5단계: 최종 완성 및 최적화
  - 단계별 프롬프트의 다섯 번째 프롬프트(Step5)를 복사해 Gemini CLI에 입력

```
Step 5: 최종 완성 및 최적화
앱을 최종 완성하고 사용성을 극대화해줘.
요구사항:
1. 데이터 내보내기/가져오기:
    • JSON 형식으로 데이터 내보내기 버튼
    • 파일 업로드로 데이터 가져오기
    • 가져오기 전 현재 데이터 백업 확인
2. 정렬 옵션:
    • 생성일순, 카테고리순, 완료 상태순 정렬
    • 정렬 상태 저장
    • 드래그 앤 드롭으로 수동 정렬 (sortable)
3. 성능 최적화:
    • 100개 이상 항목에서도 부드러운 성능
```

5단계 [내보내기] 기능 테스트

- [Export]를 누르면 애플리케이션이 데스크톱의 다운로드 경로에 JSON 파일로 저장됨



Gemini CLI 종료

- /quit 명령으로 종료

```
Shift+Tab to accept edits                                   1 GEMINI.md file

> /quit

quit   Exit the cli
workspace (/directory)                sandbox                          /model
D:\VibeCoding\Study02\Lab2            no sandbox                 Auto (Gemini 3)
```

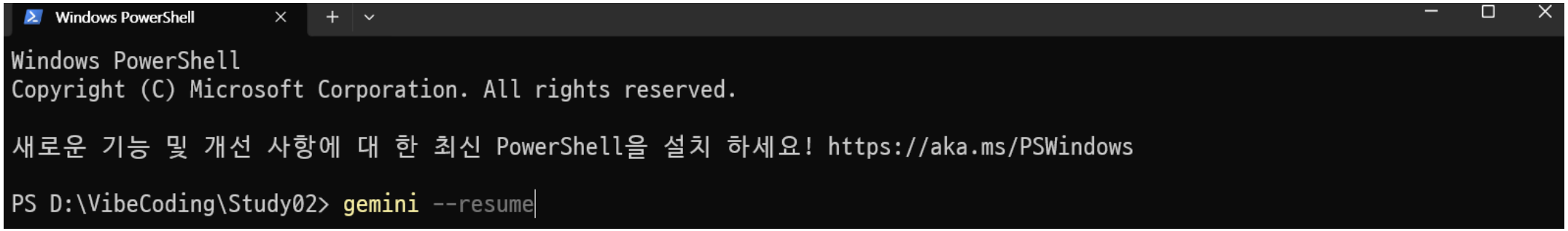
3) 작업 재개와 효율성 높이기

세션(session)

- 컴퓨터나 프로그램에서 사용자와 시스템이 연결되어 있는 일정한 기간
- Gemini CLI에서 대화를 시작해 명령을 주고받는 동안 유지되는 상태
- Gemini CLI 는 터미널을 닫으면 이전 대화가 사라지는 것처럼 보이기 때문에, 장기 프로젝트를 진행할 때는 세션과 컨텍스트 관리에 특히 주의가 필요함

세션 재개(1)

- gemini --resume 명령
  - Gemini CLI는 모든 상호작용을 자동으로 저장합니다. 별도의 저장 명령 없이 resume 기능을 사용할 수 있습니다.
  - 마지막으로 진행했던 세션, 즉 대화나 작업을 이어서 시작할 때 사용됩니다.
  - 이 기능은 며칠에 걸쳐 진행되는 코딩 작업이나 긴 컨텍스트가 필요한 대화를 다룰 때 유용합니다.
  - 활용 예시: 네트워크 문제나 터미널 종료 후 다시 작업을 시작할 때, 이전 맥락을 모두 기억하여 작업을 이어나갈 수 있습니다.



세션 재개(2)

- 세션 목록 및 전환: Gemini CLI 실행 중 /resume 명령어를 입력하면 저장된 세션 목록을 볼 수 있고, 이전 세션으로 전환할 수 있습니다.

```
> /resume

Chat Sessions (3 total)                                          sorted by date desc
Navigate: ↑/↓   Resume: Enter   Search: /   Delete: x   Quit: q
Sort: s         Reverse: r      First/Last: g/G

  Index | Msgs | Age | Name
> #1    | 81   | 15m | Implement a to-do list app with advanced features. (current)
  #2    | 6    | 2h  | 프로젝트 기술 스택 및 저장된 대화 요약
  #3    | 12   | 13d | You are an AI agent that brings the power of Gemini directly into the terminal. Your task…
```

대화 히스토리 관리

- /chat 명령으로 대화히스토리를 관리 할 수 있습니다.

```
> /chat

Chat Sessions (3 total)                                                    sorted by date desc
Navigate: ↑/↓   Resume: Enter   Search: /   Delete: x   Quit: q
Sort: s         Reverse: r      First/Last: g/G

  Index | Msgs | Age | Name
> #1    | 93   | 36m | Implement a to-do list app with advanced features. (current)
  #2    | 6    | 3h  | 프로젝트 기술 스택 및 저장된 대화 요약
  #3    | 12   | 13d | You are an AI agent that brings the power of Gemini directly into the terminal. Your task…
▼
```

자동승인 하기

- 다음 프롬프트 입력 후 자동승인(Allow for this session) 하기

```
할 일 관리 앱 "My Tasks"에 키워드 기반 자동 카테고리 분류 기능을 추가해 줘
```

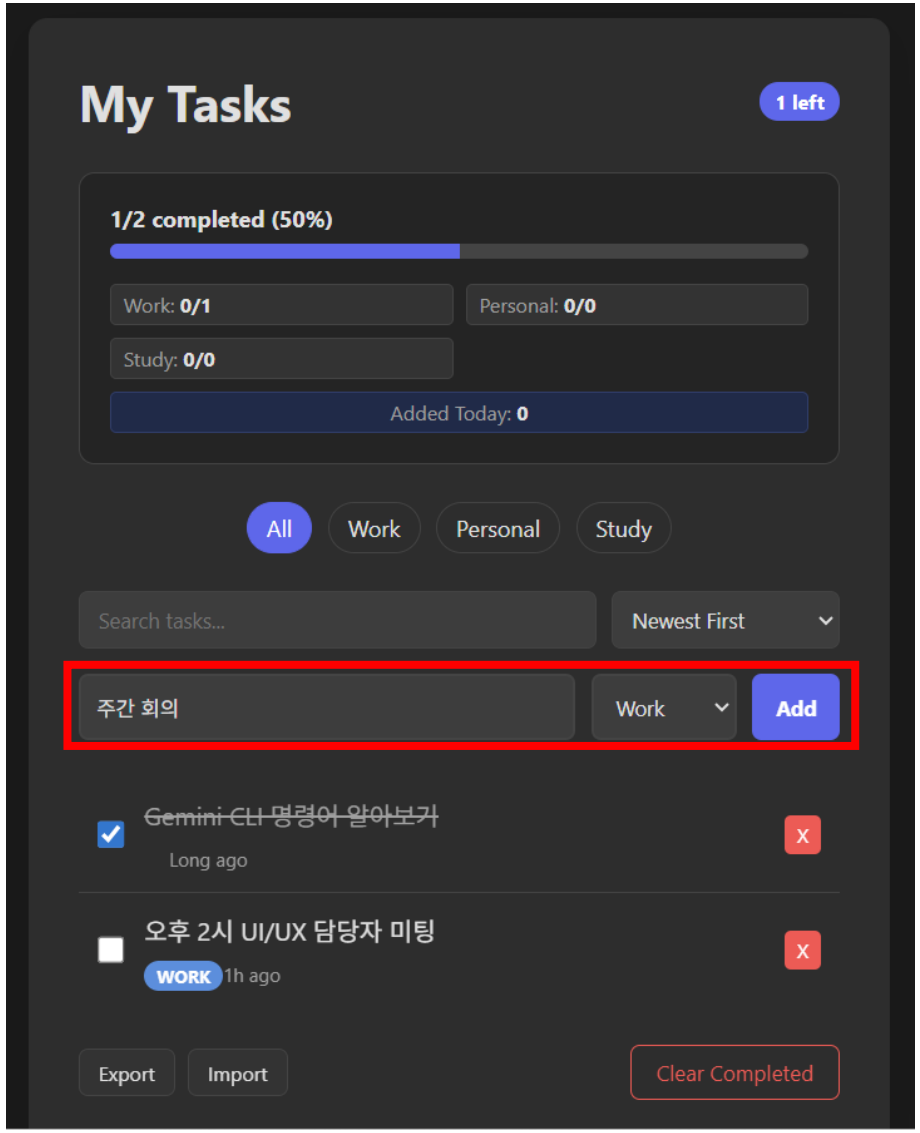
```
> 할 일 관리 앱 "My Tasks"에 키워드 기반 자동 카테고리 분류 기능을 추가해 줘

Action Required

?  Enter Plan Mode Designing keyword-based automatic category classification for the To-Do app.

This will restrict the agent to read-only tools to allow for safe planning.
Do you want to proceed?

  1. Allow once
● 2. Allow for this session
  3. No, suggest changes (esc)
```

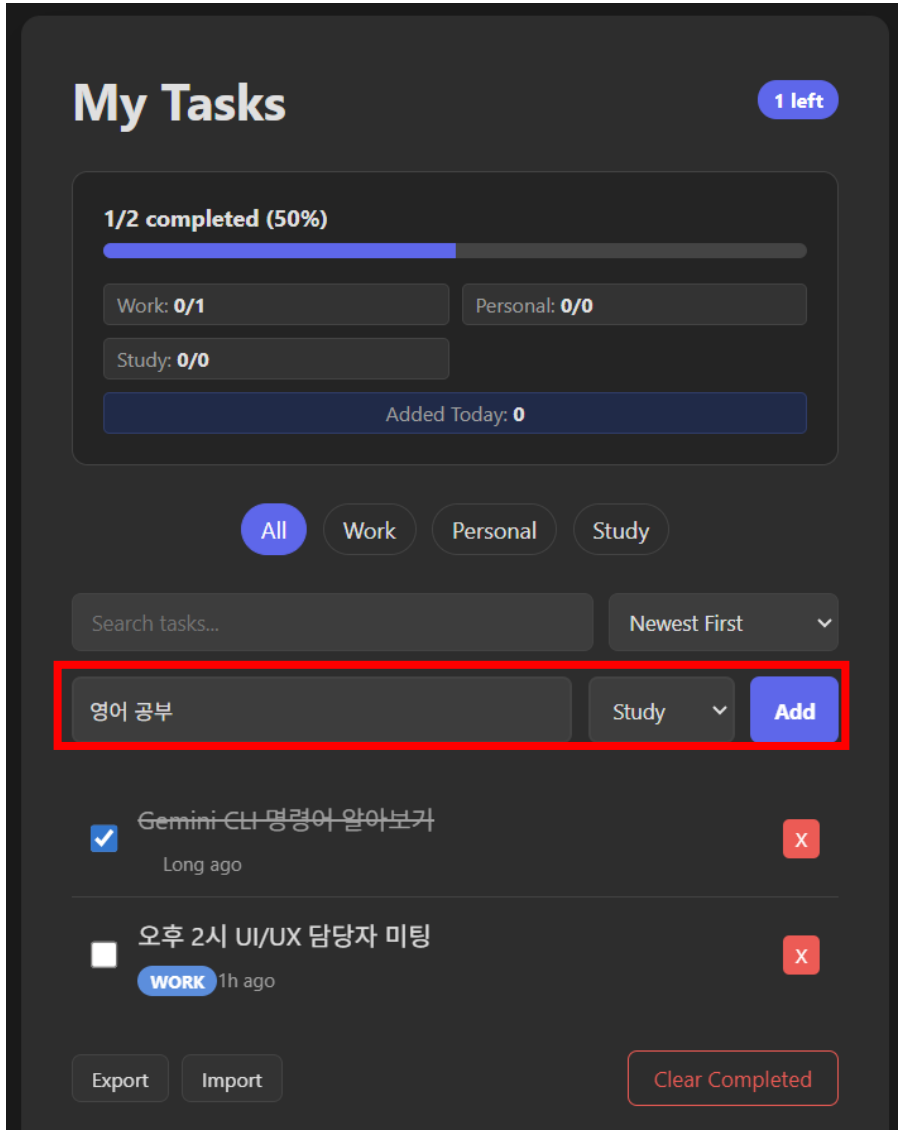
수정 작업 완료

- 작업이 완료되면 기능이 성공적으로 추가되었다는 메시지 확인 가능

```
✦ 키워드 기반 자동 카테고리 분류 기능을 추가하기 위한 전략입니다.

  이 기능은 사용자가 할 일을 입력하는 도중에 특정 키워드가 감지되면, 카테고리 선택 상자를 자동으로 해당 카테고리로
  변경해주는 편의 기능입니다. 사용자는 자동 변경된 카테고리를 그대로 쓰거나, 원한다면 추가 전 수동으로 다시 바꿀 수
  있습니다.

  주요 구현 내용 (script.js 수정):

   1. 키워드 매핑 정의: 각 카테고리별로 자주 쓰이는 단어들을 미리 정의합니다.
       * 업무 (Work): '회의', '업무', '보고', '프로젝트', '회사', '미팅', '출장', 'meeting', 'work' 등
       * 개인 (Personal): '운동', '쇼핑', '청소', '약속', '휴식', '장보기', '가족', '친구', 'shopping', 'personal' 등
       * 공부 (Study): '공부', '강의', '시험', '독서', '학습', '과제', '복습', '숙제', 'study', 'exam' 등
   2. 실시간 감지 로직: 할 일 입력창(todoInput)에 input 이벤트를 리스너를 추가하여, 사용자가 글자를 타이핑할 때마다 위
      키워드 목록에 해당하는 단어가 있는지 검사합니다.
   3. 자동 분류: 매칭되는 키워드가 발견되면 즉시 카테고리 드롭다운(categorySelect)의 값을 해당 카테고리로 변경합니다.

  이 방향으로 기능을 추가하고 GEMINI.md에도 해당 내용을 업데이트할 계획입니다. 괜찮으시다면 바로 구현 계획을 세우고
  코드를 수정하겠습니다!
```

기능 추가 확인


My Tasks
1 left
1/2 completed (50%)
Work: 0/1
Personal: 0/0
Study: 0/0
Added Today: 0
All
Work
Personal
Study
Search tasks...
Newest First
주간 회의
Work
Add
Gemini CLI 명령어 알아보기
Long ago
오후 2시 UI/UX 담당자 미팅
WORK
1h ago
Export
Import
Clear Completed


My Tasks
1 left
1/2 completed (50%)
Work: 0/1
Personal: 0/0
Study: 0/0
Added Today: 0
All
Work
Personal
Study
Search tasks...
Newest First
영어 공부
Study
Add
Gemini CLI 명령어 알아보기
Long ago
오후 2시 UI/UX 담당자 미팅
WORK
1h ago
Export
Import
Clear Completed

스테이징 확인

- /stats 명령을 통해 현재 세션 상태를 수시로 체크하는 습관이 중요합니다.
  - stats 결과는 현재 **Gemini Code Assist** 세션의 활동량과 성능, 그리고 모델 사용 현황을 종합적으로 보여줍니다.
    - Interaction Summary (상호작용 요약)
    - Performance (성능)
    - Model Usage (모델 사용 현황)

```
> /stats

Session Stats

Interaction Summary
Session ID:                 ee396715-6dfd-4933-a0e4-e3e9425c199d
Auth Method:                Signed in with Google (ciel706@gmail.com)
Tier:                       Gemini Code Assist in Google One AI Pro
Tool Calls:                 6 ( ✓ 6 x 0 )
Success Rate:               100.0%
User Agreement:             100.0% (6 reviewed)
Code Changes:               +51 -0

Performance
Wall Time:                  18m 43s
Agent Active:               5m 40s
  » API Time:               1m 23s (24.5%)
  » Tool Time:              4m 16s (75.5%)

Model                    Reqs    Model usage                Usage resets
gemini-2.5-flash-lite       1    ▬                     2%   10:03 PM (20h 53m)
gemini-3-flash-preview      6    ▬                     8%   10:03 PM (20h 53m)
gemini-3.1-pro-preview      3    ▬                    12%   11:13 PM (22h 3m)
gemini-2.5-flash            -    ▬                     8%   10:03 PM (20h 53m)
gemini-2.5-pro              -    ▬                    12%   11:13 PM (22h 3m)
gemini-3.1-flash-lite-…     -    ▬                     2%   10:03 PM (20h 53m)
```

Gemini CLI 컨텍스트

- 컨텍스트: Gemini가 현재 세션에서 기억하고 있는 소스 코드, 파일 구조, 이전 대화 기록 전체를 의미합니다.
  - Gemini CLI에서 컨텍스트가 가득 차면 AI의 반응이 느려지거나 이전 내용을 잊어버릴 수 있습니다. ➔ 이를 방지하기 위해 수동으로 컨텍스트를 압축(Compact)하거나 정리하는 것은 매우 중요합니다.
- 컨텍스트 윈도우(Context Window: LLM(대형 언어 모델)이 한 번에 이해하고 처리할 수 있는 최대 토큰(단어/문자)의 양으로, AI의 **단기 기억 용량**을 뜻합니다.
  - 시스템 프롬프트, 이전 대화 내용, 현재 입력이 모두 포함되며, 이 한계 내에서 문맥을 파악합니다.

Gemini CLI 컨텍스트 관리 명령

- /compress: 대화의 주요 내용을 유지하면서 토큰 사용량을 줄여 컨텍스트 윈도우를 최적화 합니다.
  - Gemini 2.5 Pro는 약 100만 토큰의 컨텍스트 윈도우를 제공하지만, 긴 세션에서는 /compress 명령어를 사용하여 컨텍스트 윈도우를 관리해야 합니다.

```
> /compress

✦ Chat history compressed from 89518 to 9626 tokens.
```

    - 약 **90%에 가까운 토큰을 절약**했습니다.

- /clear: 컨텍스트 윈도우를 완전히 비우고 새로운 대화를 시작합니다.

4) 프로젝트 개선과 작업관리

멀티모달 AI(multimodal AI)

- 하나의 AI가 여러 종류의 정보를 함께 이해하고 처리하는 AI
- 멀티모달 AI는 텍스트나 코드뿐만 아니라 이미지, 음성 등 서로 다른 형태의 데이터를 함께 인식할 수 있음
- Gemini CLI는 Google의 멀티모달 AI 모델인 Gemini를 기반으로 하는 터미널(명령줄) 인터페이스 도구입니다.
- Gemini CLI는 텍스트뿐만 아니라 이미지, 비디오, 오디오 등 다양한 형태의 데이터를 분석하고 처리할 수 있는 기능을 제공합니다.

주요 멀티모달 기능

- **이미지 분석(Vision):** 오류 스크린샷, UI 목업, 사진 등을 입력받아 내용을 분석하고, 코드를 생성하거나 수정 제안을 할 수 있습니다.
- **문서 요약 및 처리(PDF, TXT):** 긴 기술 명세서(PDF)나 코드 파일을 입력받아 요약하거나 핵심 내용을 추출할 수 있습니다.
- **동영상/오디오 이해:** 비디오 튜토리얼을 입력받아 내용을 분석하거나, 음성 데이터를 처리할 수 있습니다.
- **멀티모달 임베딩:** 텍스트, 이미지, 오디오, 비디오를 하나의 임베딩 공간으로 매핑하는 기능을 활용할 수 있습니다

멀티모달 기능 사용 예시

- Gemini CLI는 터미널 환경에서 gemini 명령어를 사용하여 대화형으로 멀티모달 입력을 처리합니다.
  - **이미지 분석 요청:**

```
gemini "이 스크린샷의 에러 원인을 분석하고 코드를 수정해줘" --image error_screenshot.png
```

  - 문서 요약 요청

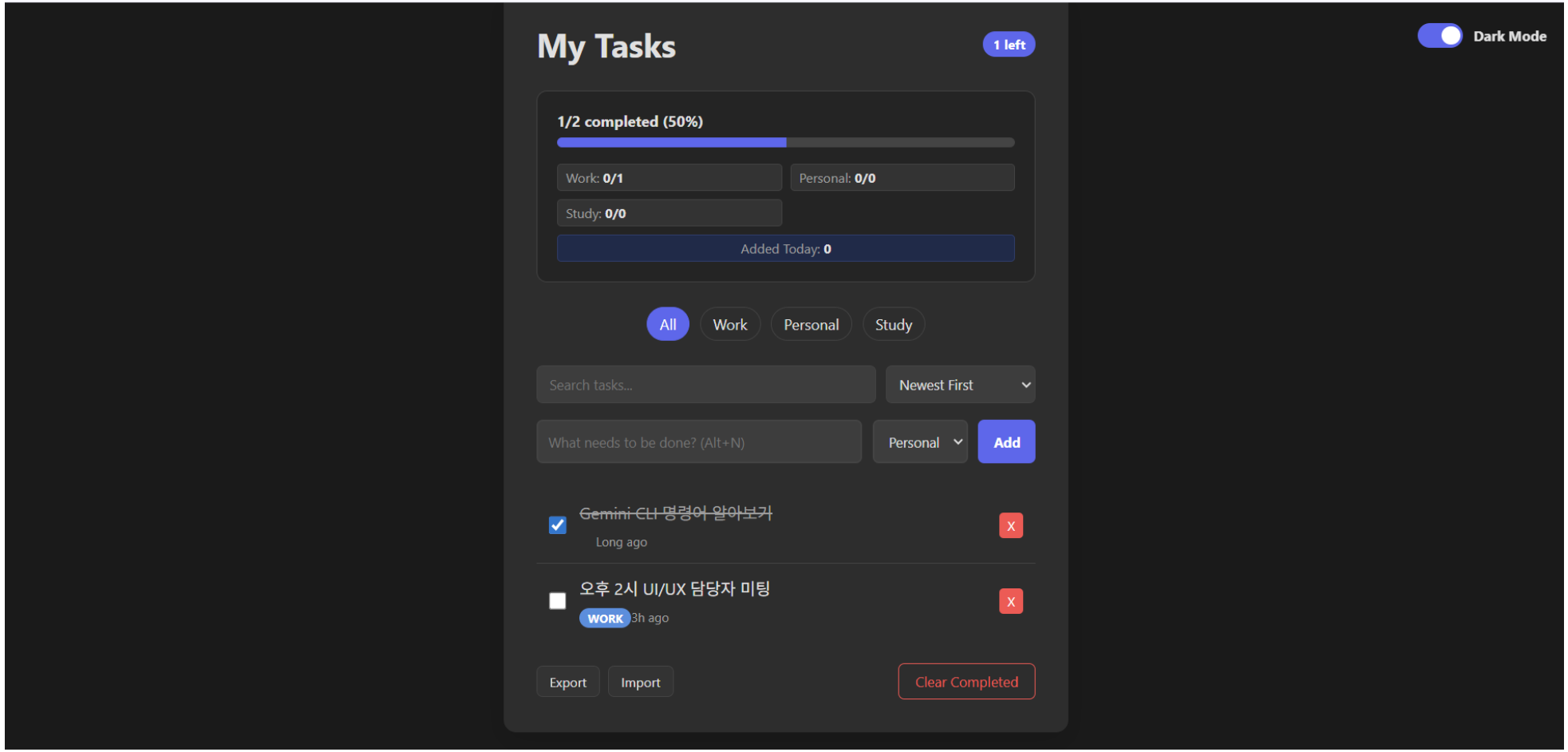
```
gemini "이 PDF 파일의 핵심 내용을 요약해줘" --file document.pdf
```

캡처 이미지로 데스크톱 버전 앱의 UI 개선하기

- 화면을 직접 캡처(capture)해 전달하면 Gemini CLI가 요구사항을 더 정확하고 빠르게 이해
- 요청할 사항
- 현재 결과가 모바일 중심의 디자인인데 변경하여 PC 버전으로 만들기
- 이전 결과를 보관하기 위해 ‘web_version’이라는 폴더를 새로 만들어 그곳에 저장하기

Gemini CLI에 이미지 전달하기

- 앱 화면을 캡처하여 Lab4 폴더에 화면캡처.png 파일로 저장



Gemini CLI에 이미지를 입력하는 방법

- '화면캡처.png' 파일을 드래그해서 클로드 코드가 실행 중인 명령 프롬프트 창에 드롭
- 이미지가 정상적으로 입력되면 프롬프트 입력창에 ‘[Image 화면캡처.png]’ 메시지 표시

```
auto-accept edits Shift+Tab to plan

> [Image 화면캡처.png]

workspace (/directory)                                  sandbox
D:\VibeCoding\Study02\Lab2                              no sandbox
```

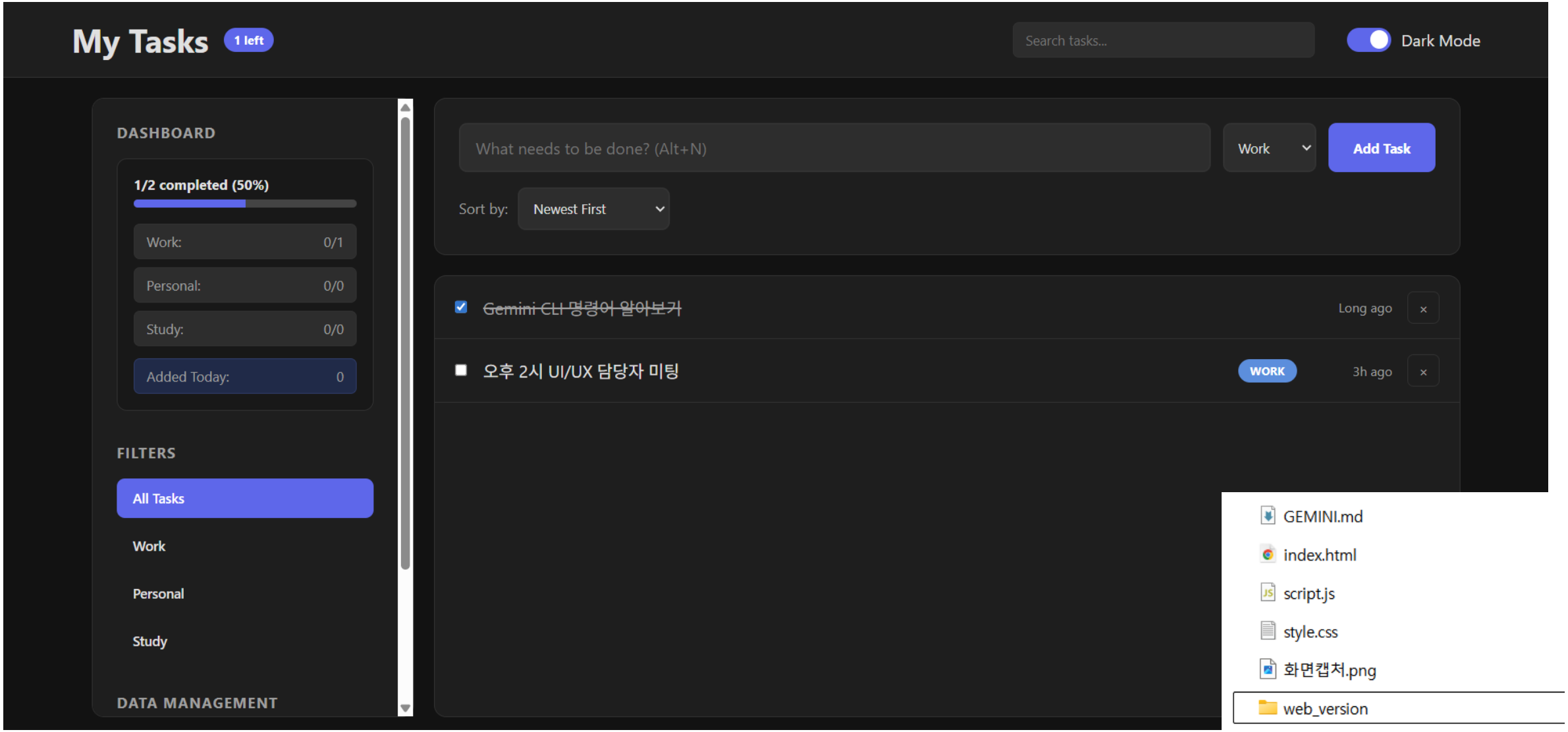
UI 개선하기

- 디자인 개선을 위해 요청 사항으로 프롬프트 입력

> 현재 디자인은 모바일에 최적화 되어 있는것 같아. 데스크톱 환경에서도 전체 화면에서 UI를 볼 수 있도록 디자인을 수정해줘. 수정한 디자인은 'web_version'이라는 새로운 폴더를 생성한 뒤 그 경로에 저장부탁해

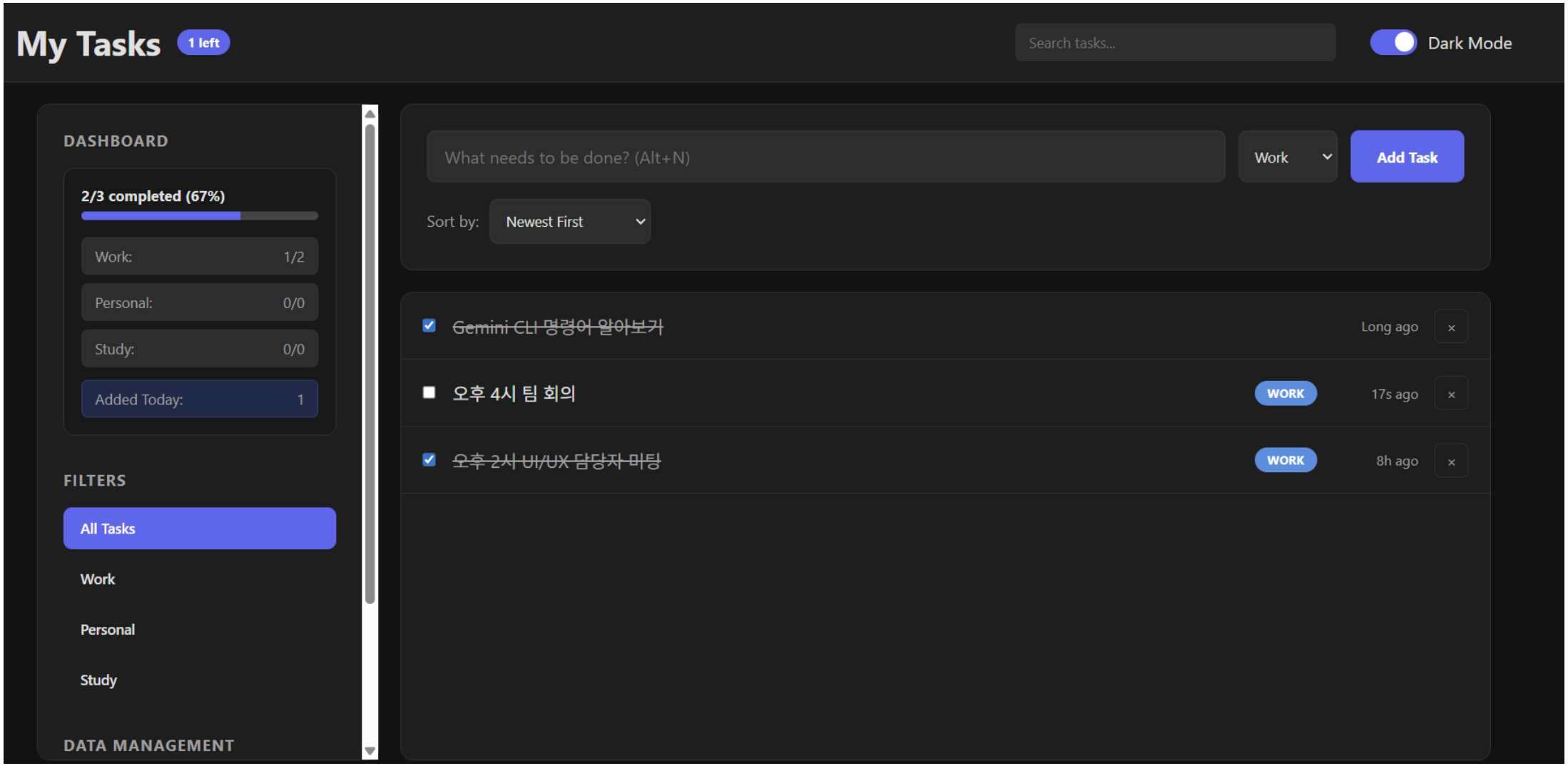
```
auto-accept edits Shift+Tab to plan                                  1 GEMINI.md file

> [Image 화면캡처.png] 현재 디자인은 모바일에 최적화 되어 있는것 같아. 데스크톱 환경에서도 전체 화면에서 UI를 볼 수
  있도록 디자인을 수정해줘. 수정한 디자인은 'web_version'이라는 새로운 폴더를 생성한 뒤 그 경로에 저장부탁해

workspace (/directory)                sandbox                                  /model
D:\VibeCoding\Study02\Lab2            no sandbox                          Auto (Gemini 3)
```

변경된 UI 확인


My Tasks
1 left
Search tasks...
Dark Mode
DASHBOARD
1/2 completed (50%)
Work: 0/1
Personal: 0/0
Study: 0/0
Added Today: 0
FILTERS
All Tasks
Work
Personal
Study
DATA MANAGEMENT
What needs to be done? (Alt+N)
Work
Add Task
Sort by:
Newest First
Gemini CLI 명령어 알아보기
Long ago
오후 2시 UI/UX 담당자 미팅
WORK
3h ago

GEMINI.md
index.html
script.js
style.css
화면캡처.png
web_version

기능확인


My Tasks
1 left
Search tasks...
Dark Mode
DASHBOARD
2/3 completed (67%)
Work: 1/2
Personal: 0/0
Study: 0/0
Added Today: 1
FILTERS
All Tasks
Work
Personal
Study
DATA MANAGEMENT
What needs to be done? (Alt+N)
Work
Add Task
Sort by:
Newest First
Gemini CLI 명령어 알아보기
Long ago
오후 4시 팀 회의
WORK
17s ago
오후 2시 UI/UX 담당자 미팅
WORK
8h ago

참조를 통해 코드 빠르게 점검하기

- @ 기호를 사용해 작업 폴더에 생성된 ‘script.js’를 불러와, 앞서 만든 할 일 관리 앱의 자바스크립트 코드를 분석하고 기능이 제대로 작동하는지 점검

@ 기호로 코드 분석하기

- Gemini CLI에서 '@'를 입력하면 나타나는 불러올 수 있는 파일 목록 중에서 web_version 경로의 'script.js' 파일을 선택

```
> @

codebase_investigator [Agent]
cli_help [Agent]
generalist [Agent]
web_version/
GEMINI.md
index.html
script.js
style.css
```

@ 기호로 코드 분석하기

- 자바스크립트 파일에서 다크 모드 기능이 잘 작성되어 있는지 확인
- 다음과 같이 프롬프트를 입력 Gemini CLI가 ‘script.js’ 파일 전체를 읽고 즉시 분석

```
@script.js 파일에서 다크 모드 관련 함수를 찾아서 설명해 줘
```

```
> @script.js 파일에서 다크 모드 관련 함수를 찾아서 설명해 줘

✓  ReadManyFiles Will attempt to read and concatenate files using patterns:

ReadManyFiles Result (Target Dir: D:\VibeCoding\Study02\Lab2)

Successfully read and concatenated content from 1 file(s).

Processed Files:
 - script.js

⠼ Thinking... (esc to cancel, 4s)                                  ? for shortcuts
```

@ 기호로 코드 분석하기 결과 확인

✦ script.js 파일에서 다크 모드(Dark Mode)를 관리하는 핵심 로직은 Theme Management 섹션에 구현되어 있으며, 크게 두 부분으로 나뉩니다.

1. 초기 테마 설정 (새로고침 시 유지)

```
1 const savedTheme = localStorage.getItem('theme') || 'light'; // 저장된 테마 불러오기
2 if (savedTheme === 'dark') {
3     document.body.classList.add('dark-theme'); // body에 dark-theme 클래스 추가
4     themeCheckbox.checked = true; // 토글 스위치를 '켜짐' 상태로 변경
5 }
```

* 설명: 사용자가 이전에 설정한 테마를 브라우저(localStorage)에서 기억했다가, 앱을 다시 열었을 때 그 설정을 그대로 적용합니다. 만약 'dark'로 저장되어 있다면 CSS의 다크 모드 스타일이 적용되도록 body 태그에 클래스를 주입합니다.

2. 실시간 테마 전환 (토글 이벤트)

```
1 themeCheckbox.addEventListener('change', () => {
2     const isDark = themeCheckbox.checked; // 체크박스 상태 확인
3     document.body.classList.toggle('dark-theme', isDark); // 클래스 토글 (추가/제거)
4     localStorage.setItem('theme', isDark ? 'dark' : 'light'); // 변경된 설정 저장
5 });
```

* 설명: 화면 우측 상단의 다크 모드 스위치를 클릭할 때 실행됩니다.
  * classList.toggle: 스위치가 켜지면 dark-theme 클래스를 body에 추가하고, 꺼지면 제거하여 즉각적으로 디자인을 변경합니다.
  * localStorage.setItem: 변경된 상태를 저장하여 다음 접속 시에도 유지되도록 합니다.

이 로직은 style.css에 정의된 --bg-color, --text-color 등의 변수값들을 한 번에 교체하는 방식으로 작동하여 매우 빠르고 부드럽게 화면을 전환합니다.

@ 기호로 폴더 구조 확인

- 클로드 코드에 '@web_version'을 입력해서 'web_version' 폴더 전체를 불러와서 web_version' 폴더에 들어 있는 하위 파일 리스트가 보임

```
> @web_version/

web_version/
web_version/script.js
web_version/style.css
web_version/index.html
```

데스크톱 버전의 최적화 확인

- 할 일 관리 앱이 데스크톱 환경에 최적화되었는지 확인하기 위해 다음과 같이 프롬프트를 입력

```
@web_version/ 폴더에 있는 파일들을 분석해서 할 일 관리 앱이 데스크톱 환경에 최적화 되어있는지
확인해 줘
```

```
web_version 폴더의 파일들을 분석한 결과, 해당 할 일 관리 앱은 데스크톱 환경에 매우 효과적으로 최적화되어 있음을 확인했습니다.
주요 분석 포인트는 다음과 같습니다.

1. 구조적 최적화 (HTML)
 * 2컬럼 레이아웃 적용: 기존의 단일 컬럼 구조에서 벗어나, <aside class="sidebar">와 <section class="content">로 분리된 사이드바
   구조를 채택했습니다. 이는 넓은 데스크톱 화면에서 시선 분산을 막고 정보를 효율적으로 배치하는 전형적인 웹 대시보드 형태입니다.
 * 시맨틱 태그 활용: <header>, <main>, <aside>, <section> 등 시맨틱 태그를 사용하여 구조가 명확하며 유지보수가 용이합니다.

2. 시각적 최적화 (CSS)
 * 화면 너비 제한 및 중앙 정렬: max-width: 1400px와 margin: 0 auto를 설정하여, 초광폭 모니터에서도 콘텐츠가 너무 양옆으로
   벌어지지 않고 보기 좋게 중앙에 위치하도록 설계되었습니다.
 * 독립적 스크롤 영역: height: 100vh와 overflow: hidden을 사용하고, 사이드바와 리스트 영역(list-container)에 각각 overflow-y:
   auto를 부여했습니다. 이를 통해 페이지 전체가 스크롤되는 대신, 특정 영역만 스크롤되는 "애플리케이션 느낌"의 UI를 구현했습니다.
 * 가로 배치 활용: 리스트 아이템(.todo-content) 내부에서 할 일 텍스트와 메타데이터(카테고리 태그, 시간)를 justify-content:
   space-between으로 양끝에 배치하여 가로 공간을 충분히 활용했습니다.

3. 기능 및 사용자 경험 최적화 (JS)
 * 키보드 단축키 유지: 데스크톱 사용자가 선호하는 Alt+N(입력창 포커스), Alt+1~4(필터 전환) 단축키가 그대로 유지되어 생산성이
   높습니다.
 * 드래그 앤 드롭: 마우스를 사용하는 데스크톱 환경에서 직관적으로 순서를 바꿀 수 있는 HTML5 Drag and Drop API가 적용되어
   있습니다.
 * 실시간 검색창 상단 배치: 접근성이 가장 좋은 헤더 영역에 검색창을 배치하여 대량의 할 일 중에서도 원하는 항목을 빠르게 찾을 수
   있습니다.

결론
현재 web_version은 모바일의 단순한 확장이 아니라, 데스크톱의 넓은 화면과 정교한 마우스/키보드 조작 환경을 완벽히 고려하여
재설계된 버전입니다. 사용자는 마치 전문적인 업무용 대시보드를 사용하는 듯한 쾌적한 경험을 할 수 있을 것으로 판단됩니다.
```

/settings 명령

- UI 환경, 키 바인딩, 접근성 기능 등 모든 Gemini CLI 설정을 조회하고 수정할 수 있는 대화 상자를 엽니다.
- 이 설정은 주로 settings.json 파일에 저장되며, 사용자 경험을 개인화하는 데 사용됩니다.

```
> /settings

> Settings

Search to filter

▲
  Output Format                                                    Text
  The format of the CLI output. Can be `text` or `json`.

  Auto Theme Switching                                             true
  Automatically switch between default light and dark themes based on terminal background color.

● Terminal Background Polling Interval                             60
  Interval in seconds to poll the terminal background color.
```

Gemini CLI의 설정 파일

- 도구의 동작을 제어하는 핵심 파일이며, 주로 홈 디렉터리의 ~/.gemini/settings.json 경로에 위치합니다.(전역설정)
- 주요 설정 내용 및 구조
  - settings.json 파일은 JSON 형식을 따르며, 다음 항목들을 포함할 수 있습니다:
    - theme: CLI 인터페이스의 테마 설정 (예: "Default").
    - model: 사용할 제미나이 모델 버전 지정 (예: gemini-1.5-pro 등).
    - api_key: Google AI Studio API 키 설정.
    - context_window: 모델이 기억할 대화 컨텍스트 크기 설정

Gemini CLI 설정파일의 관리

- 수정 방법:
  - vim ~/.gemini/settings.json 명령어를 사용하여 파일을 직접 수정할 수 있습니다.
- 전역 설정 또는 프로젝트별 설정:
  - 전역 설정 외에도 특정 프로젝트 폴더 내에 settings.json을 두어 해당 프로젝트에만 적용되는 설정을 관리할 수 있습니다

Gemini CLI Setting 우선순위

- 우선순위
  1) 프로젝트 설정: 실행하는 프로젝트 디렉토리 내에 위치한 설정 파일.
  2) 사용자/글로벌 설정: 홈 디렉토리에 위치한 전역 설정 파일.
- 위치:
  - 글로벌 설정 (~/.gemini/settings.json): 사용자 환경 전체에 적용되는 기본 설정입니다.
  - 프로젝트 설정 (./.gemini/settings.json): 특정 프로젝트에서 실행할 때만 적용되며, 글로벌 설정의 내용을 덮어씁니다.
- GEMINI.md: 일부 환경에서는 GEMINI.md 파일을 통해 프로젝트별로 더 세분화된 설정을 지원합니다.

설정 파일 위치 및 우선순위

우선순위	위치 구분	경로 (윈도우 기준 예시)	특징
1위	명령어 인자	--config 플래그 사용 시	특정 실행 시에만 임시로 위치를 지정할 때 사용
2위	로컬 프로젝트	(현재 작업 디렉터리)\.gemini\settings.json	특정 코딩 프로젝트 폴더마다 다른 설정을 적용하고 싶을 때 사용
3위	사용자 홈 (Global)	C:\Users\<사용자>\.gemini\settings.json	현재 질문하는 위치이며, 시스템 전체에서 기본으로 사용하는 전역 설정
4위	환경 변수	GEMINI_CONFIG_HOME 등	사용자가 환경 변수로 경로를 직접 지정한 경우

작동 원리 (병합 방식)

- Gemini CLI는 단순히 하나의 파일만 읽는 것이 아니라, 다음과 같은 방식으로 작동합니다.
  - 전역 설정 로드: 먼저 C:\Users\ciel7\.gemini\settings.json을 읽습니다.
  - 로컬 설정 덮어쓰기: 만약 현재 명령어를 입력하는 폴더(예: 개발 중인 프로젝트 폴더) 안에 .gemini\settings.json이 따로 있다면, 전역 설정보다 이 내용을 최우선으로 적용합니다.
  - 기본값 적용: 위 폴더들에 파일이 없거나 특정 항목이 비어 있다면, CLI 자체에 내장된 기본값(Default)을 사용합니다.

/memory 명령어

- 명령어는 AI 모델에 프로젝트의 코딩 표준, 페르소나 및 세부 사항과 같은 지속적인 컨텍스트를 제공하고 관리하는 데 사용됩니다.
- 주로 프로젝트 루트 디렉토리에 있는 GEMINI.md 파일을 통해 에이전트의 행동 규칙을 설정합니다.
- 주요 /memory 명령어 기능
  - /memory show: 현재 로드된 모든 메모리 내용을 표시
  - /memory add <텍스트>: 메모리에 새로운 규칙 또는 정보를 추가
  - /memory refresh: GEMINI.md 파일을 다시 읽고 메모리를 업데이트

Gemini CLI 메모리 시스템의 특징

- 지속적 컨텍스트: 세션이 종료된 후에도 저장된 내용은 GEMINI.md 파일에 남아, 다음 작업에 동일한 규칙을 적용합니다.
- 프로젝트별 메모리: 프로젝트 루트의 GEMINI.md에 저장되어, 해당 프로젝트의 코딩 표준이나 특정 라이브러리 사용법을 기억합니다.
- 토큰 절약: 대화 내용의 핵심 규칙을 GEMINI.md에 정의하여 긴 대화에서 효율적인 컨텍스트 관리를 통해 토큰 수를 절약합니다.

/docs 명령

- 브라우저를 통해 Gemini CLI의 전체 공식 문서를 열어주는 편리한 도구입니다.
- 주요기능
  - 공식 문서 즉시 실행: 터미널 환경을 벗어나지 않고도 브라우저에서 최신 가이드와 사용법 확인
  - 기능 및 설정 가이드: CLI의 모든 기능, 구성 스키마(Policy), 커스텀 서브에이전트 생성 방법 등 상세한 기술 문서 제공
  - 명령어 참조: 사용 가능한 모든 명령어와 해당 옵션들에 대한 상세 정보를 한눈에 볼 수 있음

THANK YOU!